# Zero-Shot Neural Network Evaluation With Sample-Wise Activation Patterns

Yameng Peng, *Student Member, IEEE*, Andy Song, *Member, IEEE*, Haytham M. Fayek, *Senior Member, IEEE*, Vic Ciesielski, and Xiaojun Chang, *Senior Member, IEEE*

***Abstract*—Zero-shot proxies, also known as training-free metrics, are widely adopted to reduce the computational overhead in neural network evaluation for scenarios such as Neural Architecture Search (NAS), as they do not require any training. Existing zero-shot metrics have several limitations, including weak correlation with the true performance and poor generalisation across different networks or downstream tasks. For example, most of these metrics apply only to either convolutional neural networks (CNNs) or Transformers, but not both. To address these limitations, we propose Sample-Wise Activation Patterns (SWAP), and its derivative, SWAP-Score, a novel and highly effective zero-shot metric. SWAP-Score is broadly applicable across both architecture families and task domains, demonstrating strong predictive performance in the majority of tasks. This metric measures the expressivity of neural networks over a mini-batch of samples, showing a high correlation with the neural networks' ground-truth performance. For both CNNs and Transformers, the SWAP-Score outperforms existing zero-shot metrics across computer vision and natural language processing tasks. For instance, Spearman's correlation coefficient between the SWAP-Score and CIFAR-10 validation accuracy for DARTS CNNs is 0.93, and 0.71 for FlexiBERT Transformers on GLUE tasks. Moreover, SWAP-Score is label-independent, hence can be applied at the pre-training stage of language models to estimate their performance for downstream tasks. When applied to NAS, SWAP-empowered NAS, SWAP-NAS can achieve competitive performance using only approximately 6 and 9 minutes of GPU time, on CIFAR-10 and ImageNet respectively.**

***Index Terms*—Zero-shot, training-free, performance evaluation, convolutional neural network, transformer, neural architecture search.**

Received 6 November 2024; revised 21 September 2025; accepted 26 April 2026. Date of publication 7 May 2026; date of current version 6 August 2026. This work was supported in part by New Generation Artificial Intelligence National Science and Technology Major Projection under Grant 2025ZD0123100 and in part by The National Natural Science Foundation of China (NSFC) under Grant 62573399 and Grant U25A20530. Recommended for acceptance by V. Murino. *(Corresponding author: Xiaojun Chang.)*

Yameng Peng is with the Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230052, China, and also with the School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: 1024peng@gmail.com).

Andy Song, Haytham M. Fayek, and Vic Ciesielski are with the School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: andy.song@rmit.edu.au; haytham.fayek@ieee.org; vic.ciesielski@rmit.edu.au).

Xiaojun Chang is with the Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230052, China (e-mail: xjchang@ustc.edu.cn).

Our code is available at: https://github.com/pym1024/SWAP_Universal.

Digital Object Identifier 10.1109/TPAMI.2026.3691075

## I. Introduction

PERFORMANCE evaluation of neural networks is essential in the deep learning field, spanning areas such as knowledge distillation, reinforcement learning, neural architecture search, and model pruning [1], [2], [3], [4], [5], [6]. Conventional approaches evaluate neural network performance via back-propagation training, which often leads to prohibitively high computational costs in certain research areas. In knowledge distillation, training a student model to replicate the teacher model's behaviour through back-propagation imposes considerable computational demands, especially with large teacher models [7], [8]. Similarly, AlphaGO achieved superhuman performance in the game of Go through reinforcement learning and Monte Carlo Tree Search, but this required thousands of self-play games, each with complex evaluations and back-propagation steps [9]. Neural architecture search (NAS), which aims to automatically design high-performing neural networks for specific tasks, requires substantial computational resources due to the need to evaluate numerous candidate networks [10], [11], [12]. To alleviate these costs, various alternatives have been proposed, including performance predictors, early-stopping, and weight-sharing strategies [13], [14], [15], [16], [17], [18].

An emerging approach is the utilisation of zero-shot metrics, also known as training-free or zero-cost proxies [3], [19], [20], [21], [22], [23], [24]. These metrics aim to eliminate the need for network training entirely as they exhibit either positive or negative correlations with the networks' ground-truth performance. Typically, these metrics necessitate only a few forward or backward passes with a mini-batch of input data, rendering their computational costs negligible compared to traditional network performance evaluation methods. However, zero-shot metrics face several challenges, including unreliable correlation with the network's ground-truth performance [19], [22] and limited generalisation across different types of neural networks and downstream tasks. In some cases, these metrics struggle to consistently outperform simple counterparts such as model parameters or FLOPs, which are computationally low-cost [25].

To overcome these limitations, we introduce Sample-Wise Activation Patterns (SWAP), and its derivative, SWAP-Score, a novel, broadly applicable, and highly effective zero-shot metric. The inspiration is from studies on network expressivity [5], [26], whilst addressing the aforementioned limitations.

SWAP-Score generalises well on neural networks based on piecewise linear and non-linear activation functions,

exhibiting strong correlations with the networks' ground-truth performance across various task types. This study rigorously evaluates its predictive capabilities on ReLU-based convolutional networks derived from five distinct architecture spaces — NAS-Bench-101, NAS-Bench-201, NAS-Bench-301, TransNAS-Bench-101-Micro/Macro [27], [28], [29], [30] — across eight different computer vision tasks. SWAP-Score is benchmarked against 15 existing training-free metrics for the correlation between metrics and networks' ground-truth performance. Furthermore, similar comparisons are performed on GELU-based Transformers from the FlexiBERT architecture space [31] with the General Language Understanding Evaluation (GLUE) tasks [32]. Finally, we demonstrate SWAP-Score's capability by integrating it into NAS as SWAP-NAS. This method combines the efficiency of SWAP-Score with the effectiveness of population-based evolutionary search, which is typically computationally intensive. The **primary contributions** of this work are as follows:

- We introduce a novel, broadly applicable, and highly effective zero-shot metric, SWAP-Score, based on **S**ample-**W**ise **A**ctivation **P**atterns. SWAP-Score demonstrates significantly higher capability in evaluating networks compared to state-of-the-art (SOTA) zero-shot metrics. Its robust, superior performance, and high generalisation capabilities are validated through comprehensive experiments across various types of neural networks, including convolutional neural networks (CNNs) and Transformers, spanning a range of computer vision and natural language processing tasks.
- We apply SWAP-Score to a practical scenario—neural architecture search (NAS)—which traditionally requires extensive network training. By integrating SWAP-Score with an evolutionary algorithm, we enable a highly efficient NAS algorithm, SWAP-NAS, capable of completing a search on CIFAR-10 in just 0.004 GPU days (6 minutes), outperforming state-of-the-art NAS methods in both speed and performance, as illustrated in Fig. 1. A direct search on ImageNet takes only 0.006 GPU days (9 minutes) to achieve SOTA accuracies, highlighting its exceptional efficiency and performance.

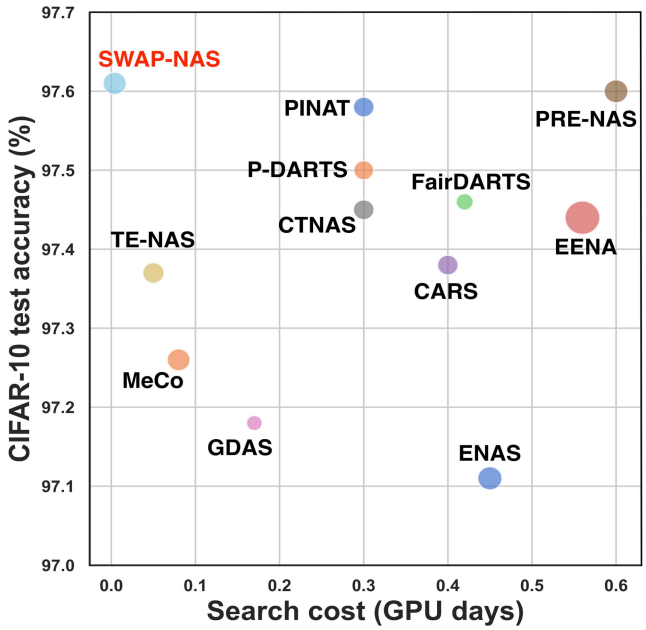


Fig. 1. Search cost and performance on CIFAR-10 for SWAP-NAS compared to state-of-the-art NAS methods. Only methods requiring less than 1 GPU day are shown. The dot size indicates model size, highlighting that SWAP-NAS attains superior performance with significantly lower search cost.

## II. Related Work

In deep learning, full evaluation by back-propagation training is the standard approach for assessing a neural network's performance on specific tasks. Neural Architecture Search (NAS) [10] is a prominent example where dense evaluations are required. For instance, AmoebaNet [33] employs an evolutionary search strategy that trains each sampled network from scratch, consuming roughly 3150 GPU days on CIFAR-10.

To mitigate this cost, subsequent studies explored few-shot or early-stop evaluation [34], [35], [36], where architectures are only trained for a small number of epochs to approximate final accuracy. Although this reduces training time, the correlation between early and final accuracy can be unreliable, especially across diverse architecture families.

A further step involves proxy evaluation, where networks are trained on simplified tasks to serve as a proxy for full evaluation [37], [38]. Examples include training on a subset of the dataset, using reduced image resolutions, or restricting model depth/width. These proxies lower cost but often fail to generalise across tasks or search spaces, and the accuracy of ranking remains limited.

To address these limitations, performance predictors were introduced. These methods fit regression models on architecture–accuracy pairs to predict unseen architectures' performance [14], [15], [38], [39], [40]. While effective once trained, this strategy requires preparing the predictor dataset itself, which demands substantial computational overhead.

Another widely used approach is weight sharing, where candidate architectures inherit parameters from a supernet rather than being trained independently [17], [18], [35], [41], [42], [43], [44]. For example, DARTS [18] combines a one-shot supernet with gradient-based optimisation, requiring only 4 GPU days to reach 97.33% test accuracy on CIFAR-10. However, parameter sharing across heterogeneous networks is problematic, and supernet optimisation is complex and difficult to generalise to broader settings [40], [45].

More recently, zero-shot metrics have emerged, eliminating the need for training altogether [3], [19], [20], [21], [22], [23], [24], [46]. For example, TE-NAS [19] combines the number of linear regions [26] with the spectrum of the neural tangent kernel [47], reducing the search cost to 0.05 GPU days on CIFAR-10 and 0.17 on ImageNet. NWOT [3] measures the overlap of activations between input samples in untrained networks, while Zen-Score [20] estimates network expressivity by

averaging Gaussian complexity from random inputs. With a specialised search space, Zen-NAS achieves 83.6% top-1 accuracy on ImageNet within 0.5 GPU day, marking the first zero-shot NAS to outperform training-based methods. MeCo [46] was proposed as a correlation-based proxy. It leverages the minimum eigenvalue of the Pearson correlation matrix across feature maps, and eliminates the need for back-propagation and data labels. Alongside CNN-oriented metrics, Transformer-specific indicators have also been introduced. TF-NAS proposed the DSS-indicator, combining synaptic diversity for self-attention with synaptic saliency for MLP modules [48], and NAS-BERT-Benchmark introduced Attention Confidence, measuring the certainty of each attention head [49].

However, an empirical study highlights the limitations of existing training-free metrics. NAS-Bench-Suite-Zero [25] evaluated 13 such methods across multiple tasks and reported that most do not generalise well across architecture families and tasks. In some cases, simple baselines such as model parameter count or FLOPs even outperform more complex metrics, motivating the need for new approaches with stronger cross-domain generalisation.

## III. Sample-Wise Activation Patterns and SWAP-Score

To address the above challenges, we introduce Sample-Wise Activation Patterns and its derivative, SWAP-Score. Similar to NWOT and Zen-Score, SWAP-Score draws inspiration from studies on network expressivity, aiming to uncover the expressivity of deep neural networks by examining their activation patterns. What sets SWAP-Score apart from other training-free metrics is its foundation on sample-wise activation patterns, which offers several benefits including high correlation with the networks' ground-truth performance and good generalisation across different types of neural networks and tasks.

The following subsections are organised as: Section III-A introduces the concept of standard activation patterns, which can reveal a network's expressivity. Section III-B, sample-wise activation patterns and SWAP-Score are presented. Section III-C introduces a regularisation approach to SWAP-Score, specifically designed for NAS scenarios.

### A. Standard Activation Patterns, Network's Expressivity & Limitations

Studies exploring the expressivity of deep neural networks [5], [26], [50] demonstrate that networks employing piecewise linear activation functions, such as ReLU [51], partition their input space into two regions: either zero or a non-zero positive value. These ReLU activation functions introduce piecewise linearity into the network. Since the composition of piecewise linear functions remains piecewise linear, a ReLU neural network can be viewed as a piecewise linear function. Consequently, the input space of such a network can be divided into multiple distinct segments, each referred to as a linear region. The number of distinct linear regions serves as an indicator of the network's functional complexity. A network with more linear regions is capable of capturing more complex features in the data, thereby exhibiting higher expressivity. Although the concept of linear region is originally defined for piecewise linear activation functions, we extend this idea to piecewise non-linear activation functions like GELU [52].

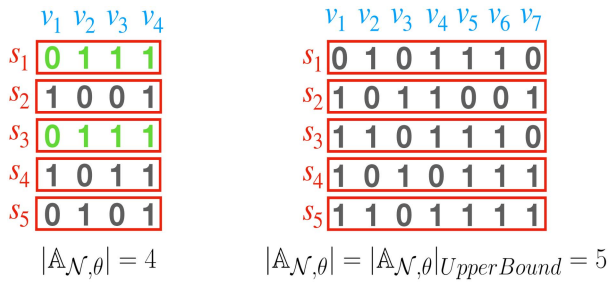


Fig. 2. Examples of activation pattern matrices $\mathbb{A}_{\mathcal{N},\theta}$ for the same network with different inputs. Duplicate activation patterns are highlighted in green.

*Definition III.1:* Given $\mathcal{N}$ as a deep neural network with piecewise linear activation function, $\theta$ as a fixed set of network parameters (randomly initialised weights and biases) of $\mathcal{N}$, a batch of inputs containing $S$ samples, the standard activation pattern, $\mathbb{A}_{\mathcal{N},\theta}$, is defined as a set of post-activation values shown as follows:

$$\mathbb{A}_{\mathcal{N},\theta} = \left\{ \mathbf{p}^{(s)} : \mathbf{p}^{(s)} = \mathbb{1}(p_v^{(s)})_{v=1}^{V}, \tilde{}s \in \{1, \ldots, S\} \right\}, \quad (1)$$

where $V$ denotes the number of intermediate values feeding into activation functions. $p_v^{(s)}$ denotes a single post-activation value from the $v^{th}$ intermediate value at $s^{th}$ sample. $\mathbb{1}(x)$ is the indicator function. We adopt the $Signum$ function as the indicator function as shown in (2).

$$\mathbb{1}(x) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases} \quad (2)$$

For ReLU-based neural networks, the indicator function converts positive non-zero values to one whilst leaving zero values unchanged. For GELU-based neural networks, the indicator function converts positive non-zero values to one, negative values to negative one and leaves zero values unchanged. Consequently, $\mathbb{A}_{\mathcal{N},\theta}$ represents a set containing the binarised or ternarised post-activation values produced by network $\mathcal{N}$ with parameters $\theta$ and $S$ input samples. This design choice is motivated by both efficiency and robustness. Continuous activation magnitudes vary substantially across different architectures. Thresholding with the Signum function provides a compact and noise-resistant representation, while also enabling efficient counting of distinct activation patterns at scale.

Following the concept discussed in [5], [26], [50], the network's expressivity can be revealed by counting the cardinality of a set composed of activation patterns.

*Limitation:* As illustrated in Fig. 2, the set $\mathbb{A}_{\mathcal{N},\theta}$ can be viewed as a matrix, with each element as a row representing a vector $\mathbb{1}(p_v^{(s)})_{v=1}^{V}$ of binarised or ternarised post-activation values over all intermediate values in $V$. Each value or cell corresponds to $\mathbb{1}(p_v^{(s)})$ as defined in (1). Since $V$ represents the number of intermediate values feeding into the activation layers, define $\mathcal{N}$

contains $L$ layers, we have:

$$\begin{cases} V_{\text{CNN}} = \sum_{l=1}^{L} \left( c_l \times \left( \lfloor \frac{w_l - k_l}{t_l} \rfloor + 1 \right) \times \left( \lfloor \frac{h_l - k_l}{t_l} \rfloor + 1 \right) \right), \\ V_{\text{Transformer}} = \sum_{l=1}^{L} T \times d_{\text{ff}}. \end{cases} \tag{3}$$

For CNNs, $c_l$ denotes the number of convolution kernels, $t_l$ denotes the stride of convolution kernels, $k_l$ denotes the kernel size, $w_l$ and $h_l$ are the width and height of the feature map in layer in $l^{th}$ layer. For Transformers, $T$ denotes the sequence length, $d_{ff}$ is the dimension of the feed-forward network's hidden layer. Hence, the length of $\mathbb{1}(p_v^{(s)})_{v=1}^{V}$ is set by the dimensionality of the input samples and depth of the network.

The upper bound of the cardinality of $\mathbb{A}_{\mathcal{N}}$ is equal to the number of input samples, $S$. Given the same number of inputs, higher-dimensional inputs or deeper networks will generate more intermediate values, i.e., $V \gg S$, making it more likely to produce distinct vectors and reach the upper bounds of cardinality. As a simplified illustration, Fig. 2 (left) shows the matrix $\mathbb{A}_{\mathcal{N},\theta}$ derived from low-dimensional inputs, while that of Fig. 2 (right) is derived from higher-dimensional inputs. Due to pattern duplication, the cardinality in the former is 4, whereas in the latter is 5. Note, the latter reaches the upper limit, the number of input samples, $S$, that is 5in this case. This highlights the limitation of examining standard activation patterns for measuring the network's expressivity. Methods leverage standard activation patterns typically require inputs of small dimensions, e.g., $1 \times 3 \times 3$ [19]. Otherwise, the metric values from different networks will all approach the number of input samples, $S$, making them indistinguishable.

### B. Sample-Wise Activation Patterns

Sample-Wise Activation Patterns address the limitation identified above. It preserves the information within the activation values, while bringing a significantly higher upper bound for the cardinality. This gives its derivative, SWAP-Score, a better capability to distinguish networks with different performance in a more subtle way.

*Definition III.2 (Sample-Wise Activation Patterns):* Given a deep neural network $\mathcal{N}$, $\theta$ as a fixed set of network parameters (randomly initialised weights and biases) of $\mathcal{N}$, a batch of inputs containing $S$ samples, sample-wise activation patterns $\hat{\mathbb{A}}_{\mathcal{N},\theta}$ is defined as follows:

$$\hat{\mathbb{A}}_{\mathcal{N},\theta} = \left\{ \mathbf{p}^{(v)} : \mathbf{p}^{(v)} = \mathbb{1}(p_s^{(v)})_{s=1}^{S}, \tilde{}v \in \{1, \ldots, V\} \right\}, \tag{4}$$

where $p_s^{(v)}$ denotes a single post-activation value from the $s^{th}$ sample at the $v^{th}$ intermediate value.

Note, in comparison with $\mathbb{A}_{\mathcal{N},\theta}$ in (1), the vectors here are now sample-wise (over neurons) rather than value-wise (over samples) as in standard activation patterns. In sample-wise activation patterns, $\mathbb{1}(p_s^{(v)})_{s=1}^{S}$ is a vector containing binarised or ternarised post-activation values across all samples in $S$.

*Definition III.3 (SWAP-Score $\Psi$):* Given a SWAP set $\hat{\mathbb{A}}_{\mathcal{N},\theta}$, the SWAP-Score $\Psi$ of network $\mathcal{N}$ with a fixed set of network parameters $\theta$ is defined as the cardinality of the set, computed as follows:

$$\mathbf{\Psi}_{\mathcal{N},\theta} = \left| \hat{\mathbb{A}}_{\mathcal{N},\theta} \right|. \tag{5}$$

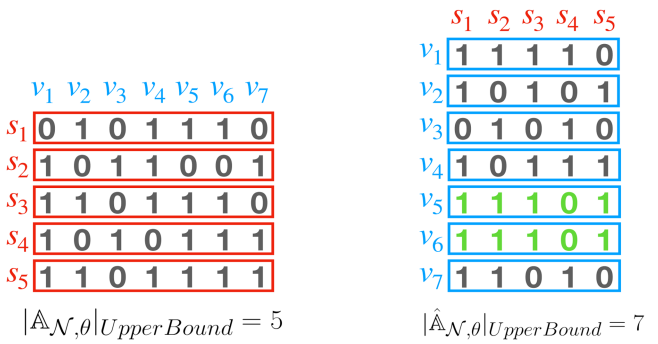


Fig. 3. Comparison between original activation pattern matrix $\mathbb{A}_{\mathcal{N},\theta}$ and sample-wise activation pattern matrix $\hat{\mathbb{A}}_{\mathcal{N},\theta}$ for the same network. Duplicate patterns are denoted in green. Sample-wise activation pattern matrix offer a higher capacity for unique patterns.

Fig. 3 illustrates the connection and the difference between $\mathbb{A}_{\mathcal{N},\theta}$ and $\hat{\mathbb{A}}_{\mathcal{N},\theta}$ in a simplified form. Both sets are based on the same network with the same input. Hence they have the same set of binarised or ternarised post-activation values but are represented differently. The upper bound of the cardinality using standard activation patterns, $\mathbb{A}_{\mathcal{N},\theta}$, is 5. In contrast, the upper bound of sample-wise activation patterns, $\hat{\mathbb{A}}_{\mathcal{N},\theta}$, is extended to 7. According to (3), the number of intermediate values $V$ grows exponentially with either an increase in the dimensionality of the input samples or the depth of the neural networks. This implies that the number of intermediate values, $V$, would be much larger than the number of input samples, $S$. As a result, SWAP has a significantly higher capacity for distinct patterns, which allows SWAP-Score to measure the network's expressivity more fine-grained. Specifically, this characteristic leads to a high correlation with the ground-truth performance of network $\mathcal{N}$ (see Section IV for more details).

### C. Regularisation

In the NAS scenarios, zero-shot metrics have a well-known drawback. They tend to favour larger models [12], meaning they do not naturally lead to smaller models when such models are desirable. Using a convolutional neural network as an example, the convolution operations with larger kernel sizes or more channels have more parameters while producing more intermediate values compared to operations like skip connection [53] or pooling layer. Consequently, larger networks typically yield higher metric values, which may not always be desirable. To mitigate this bias, we add regularisation to SWAP-Score.

*Definition III.4 (Regularisation):* Given the total number of network parameters, $\Theta$, coefficients, $\mu$ and $\sigma$, SWAP regularisation is defined as follows:

$$f(\Theta) = e^{-\left(\frac{(\Theta-\mu)^2}{\sigma}\right)}. \tag{6}$$

*Definition III.5 (Regularised SWAP-Score):* Given regularisation function $f(\Theta)$, SWAP-Score $\Psi$ of network $\mathcal{N}$ with a

fixed set of network parameters $\theta$, regularised SWAP-Score $\Psi'$ is defined as:

$$\mathbf{\Psi}'_{\mathcal{N},\theta} = \mathbf{\Psi}_{\mathcal{N},\theta} \times f(\Theta). \tag{7}$$

Regularisation function $f(\Theta)$ is a bell-shaped curve. Coefficient $\mu$ controls the centre position of this curve. Coefficient $\sigma$ adjusts the width of the curve. By explicitly setting the values for $\mu$ and $\sigma$, the regularised SWAP-Score, $\mathbf{\Psi}'_{\mathcal{N},\theta}$, can guide the resulting architectures toward a desired range of model sizes.

## IV. Experiments

Comprehensive experiments are conducted to confirm the effectiveness and generalisation of SWAP-Score. For ReLU-based networks and computer vision tasks, SWAP-Score is benchmarked against 16 other zero-shot metrics across five distinct search spaces and seven tasks (Section IV-A). For GELU-based networks and natural language processing tasks, SWAP-Score is compared with 10 zero-shot metrics, including some used in computer vision experiments and those specifically designed for Transformers and NLP tasks, on 500 BERT-like Transformers from the FlexiBERT [31]. Subsequently, the regularised SWAP-Score is integrated with evolutionary search as SWAP-NAS to evaluate its performance in a practical scenario, Neural Architecture Search (NAS), which traditionally involves a large amount of network training. Furthermore, SWAP-NAS is compared with SOTA NAS methods in terms of both search performance and efficiency (Section IV-C).

### A. SWAP-Score on ReLU-Based Networks and Computer Vision Tasks

For computer vision tasks, the networks are trained in a conventional supervised manner, which involves directly training networks on the training set until convergence and validating on the test set. The computer vision tasks are:

1) *Image classification tasks:* CIFAR-10 / CIFAR-100 [54], ImageNet-1k [55] and ImageNet16-120 [56].
2) *Object detection task:* Taskonomy dataset [57].
3) *Scene classification task:* MIT Places dataset [58].
4) *Jigsaw puzzle:* the input is divided into patches and shuffled according to preset permutations. The objective is to classify which permutation is used [25].
5) *Autoencoding:* a pixel-level prediction task that encodes images into low-dimensional latent representation then reconstructs the raw image [25].

Five architecture spaces are used to verify the effectiveness of SWAP-Score on ReLU-based networks. These include micro/cell-based search spaces, which focus on searching for optimal building blocks or cells, and macro search spaces, which focus on searching for the entire network architecture. The specific spaces are:

1) *NAS-Bench-101 [27]:* a cell-based benchmark search space which contains 423624 unique architectures trained on CIFAR-10. The architectures are designed as ResNet-like and Inception-like [53], [59].
2) *NAS-Bench-201 [28]:* a cell-based benchmark search space which contains 15625 unique architectures trained on CIFAR-10, CIFAR-100 and ImageNet16-120.
3) *NAS-Bench-301 [29]:* a surrogate benchmark space which contains architectures sampled from the DARTS search space.
4) *TransNAS-Bench-101-Mirco/Macro [30]:* consisting of a micro (cell-based) search space of size 4096, and a macro (stack-based) search space of size 3256.

SWAP-Score is compared against 15 zero-shot metrics [3], [4], [20], [21], [24], [60], [61], [62], [63], [64] across different architecture spaces and tasks, in terms of correlation. Our regularised SWAP-Score is also included in the comparison as it is used in the NAS experiments on computer vision tasks that will be presented later. The extensive experiments follow exactly the same setup as NAS-Bench-Suite-Zero [25], which is a standardised framework for verifying the effectiveness of zero-shot metrics. The hyper-parameters, such as batch size, input data, sampled architectures and random seeds are fixed and consistently applied to all these zero-shot metrics as in NAS-Bench-Suite-Zero. Specifically, the metrics' values are calculated by feeding a mini-batch of samples from each target dataset, then calculating the correlation between metrics' values and the network's ground-truth performance of each target dataset.

The comparison is shown in Fig. 4, where each column represents the Spearman coefficients of all metrics on one task with one given search space. They are computed on 1000 randomly sampled architectures. Each value in Fig. 4 is an average of five independent runs with different random seeds. For regularised SWAP-Score, the $\mu$ and $\sigma$ factors for the regularisation function are determined by the model size distribution based on 1000 randomly sampled architectures. This process requires only a few seconds for each search space.

The results in Fig. 4 clearly demonstrate the superior predictive capability of SWAP-Scores across diverse types of architectural spaces and tasks. Notably, both SWAP-Score and its regularised version outperform 15 other metrics in the majority of the evaluations. An interesting observation is a significant enhancement in SWAP-Score's performance when regularisation is applied (noted as 'reg_swap'), although its original intention is to control the model size during architecture search. This improvement is particularly evident in cell-based search spaces, including NAS-Bench-101, NAS-Bench-201, NAS-Bench-301, and TransNAS-Bench-101-Micro. However, it is worth noting that regularisation does not appear to impact the correlation results in the macro architecture space, TransNAS-Bench-101-Macro.

Figs. 5 and 6 illustrate the detailed relationships between SWAP-Score, its regularised version, and networks' ground-truth performance across each architecture space and computer vision task presented in Fig. 4. Each dot in these figures indicates a distinct ReLU-based network. From the figures, it can be observed that the regularisation makes dots more concentrated in the micro architectural spaces, i.e., NAS-Bench-101/201/301, thus leading to a higher correlation. However, in the macro architecture spaces, the distribution of the dots remains almost

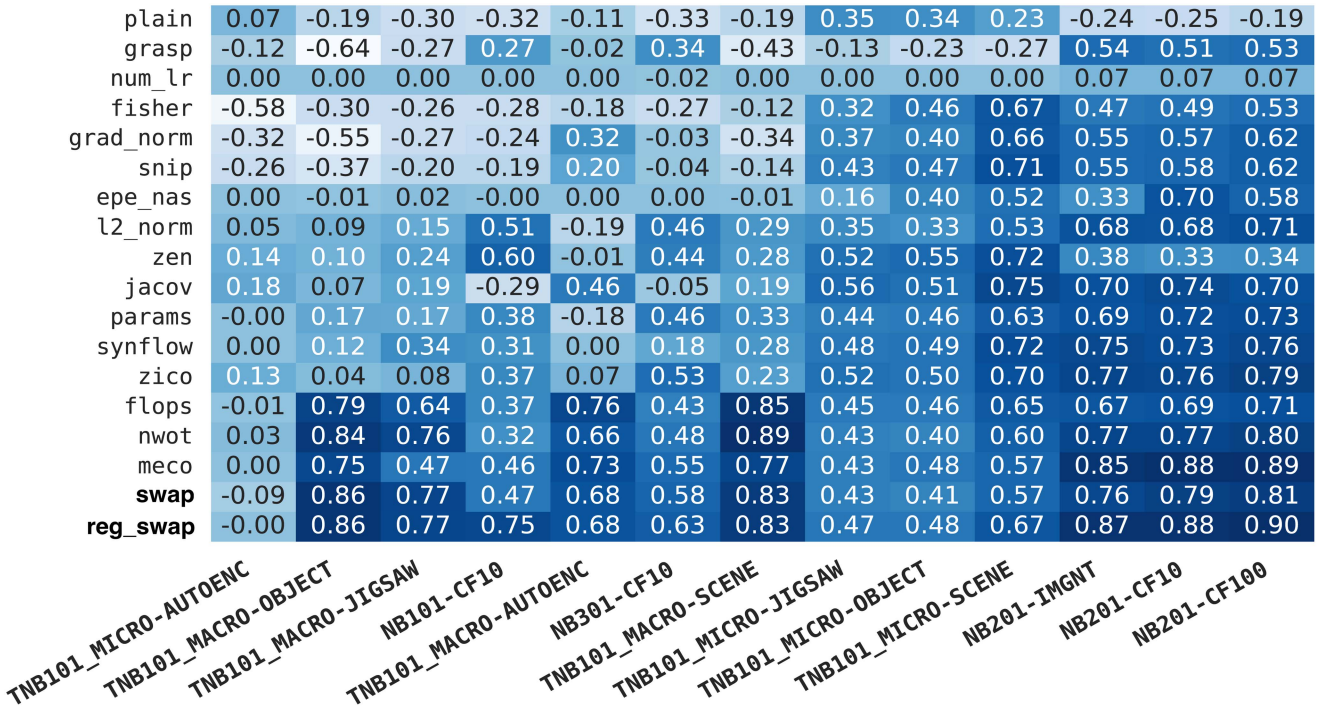


Fig. 4. Spearman's correlation coefficients between 18 training-free metrics (rows), including SWAP-Score and its regularised variant, across task - search space pairs (columns), Each cell reports Spearman's correlation coefficient between metric value and the ground-truth performance, computed over 1000 architectures randomly sampled from the given space and averaged over five independent runs. Rows and columns are ordered by mean values. SWAP-Score and its regularised variant consistently rank among the top-performing metrics across most tasks. The colour scale denotes coefficients in $[-1, 1]$.

the same. The hypothesis is that final networks from micro architecture spaces are constructed by repeating the same building blocks/cells multiple times, thus their performance is more linear with the number of model parameters. Hence, when regularisation is applied based on model parameters to micro networks, the performance aligns more closely with the final models.

### *B. SWAP-Score on GELU-Based Networks and Natural Language Processing Tasks*

For natural language processing tasks, following the setup in NAS-BERT-Benchmark [49], we leverage 500 unique GELU-based Transformer networks which sampled from the Flex-iBERT architecture space [31] and pre-trained on the Open-WebText corpus [65], which contains approximately 40 GB of text data from webpages. Specifically, these networks follow ELECTRA-Small setup and pre-training scheme [66]. Subsequently, these 500 pre-trained Transformers are fine-tuned and evaluated on 8 NLP tasks from the General Language Understanding Evaluation (GLUE) benchmark [32]. The final GLUE score is calculated as a weighted average of the individual task scores, for each Transformer network. Specifically, the natural language processing tasks are:

1) *Corpus of Linguistic Acceptability (CoLA):* Single-sentence classification, determines whether a sentence is grammatically acceptable [67].
2) *Multi-Genre Natural Language Inference(MNLI):* Multi-class classification, determines whether a hypothesis entails, contradicts, or is neutral to a premise [68].
3) *Microsoft Research Paraphrase Corpus (MRPC):* Sentence pair classification, identifies whether two sentences are paraphrases [69].
4) *Question Natural Language Inference (QNLI):* Sentence pair classification, determines whether a sentence contains the answer to a question [70].
5) *Quora Question Pairs (QQP):* Sentence pair classification, determines whether two questions are semantically equivalent [32].
6) *Recognising Textual Entailment (RTE):* Sentence pair classification which determines whether one text entails another [32].
7) *Stanford Sentiment Treebank (SST):* Single-sentence classification, determines the sentiment of a movie review [71].
8) *Semantic Textual Similarity (STS):* Regression task, evaluate the semantic similarity of two sentences [72].

The metric calculation for natural language processing tasks differs from the experiments in computer vision tasks, as Transformers typically follow a pre-train and fine-tune approach. Thus, the goal is to avoid pre-training and directly predict the networks' performance on downstream tasks. The metrics' values are calculated by feeding a mini-batch of samples from the pre-training dataset (OpenWebText), then calculating the correlation between metrics' values and the network's ground-truth performance on the downstream task (GLUE). No labels are available during the metric calculation. Most training-free metrics designed for ReLU-based CNNs are not applicable for Transformers, as they require either target labels or specific

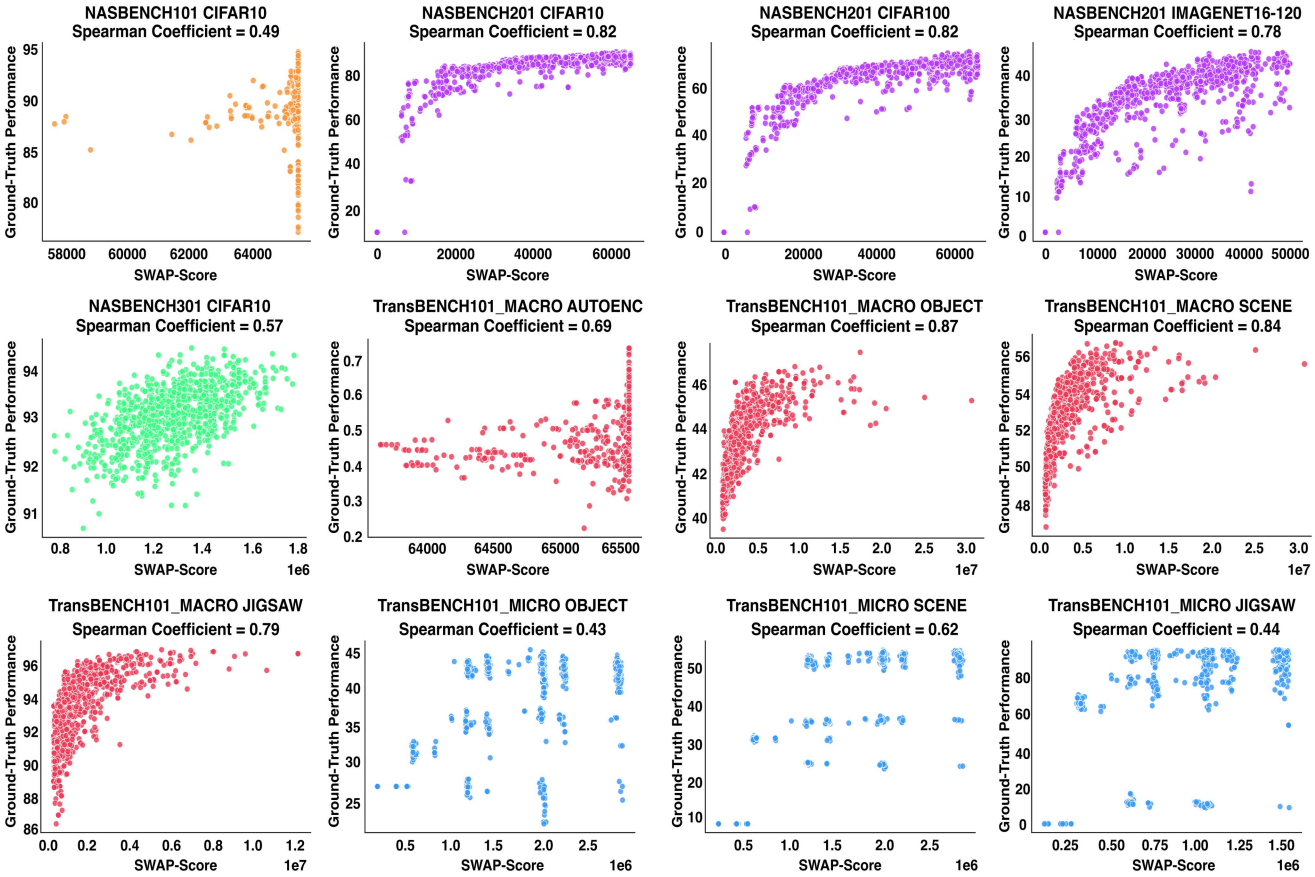


Fig. 5. Scatter plots of SWAP-Score vs. ground-truth performance for architectures from NAS-Bench-101/201/301 and TransNAS-Bench-101. Each subplot corresponds to a vision dataset, and each dot represents a single architecture. The strong correlations confirm that SWAP-Score reliably predicts performance across diverse search spaces.

TABLE I
SPEARMAN'S CORRELATION COEFFICIENTS BETWEEN 11 ZERO-SHOT METRICS (INCLUDING OUR SWAP-SCORE) AND GLUE SCORE OF 500 BERT-LIKE TRANSFORMERS

| | Synaptic Diversity | Synaptic Saliency | Activation Distance | Jacobian Score | Attention Confidence | Attention Importance | Attention Softmax Confidence | DSS-Indicator | #Params | #FLOPs | SWAP-Score |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Spearman Coefficient | 0.169 | 0.177 | 0.031 | 0.016 | 0.666 | 0.162 | 0.055 | -0.071 | 0.653 | 0.652 | **0.711** |

components of CNNs. In contrast, the SWAP-Score is label-independent and only requires activation values, thus it can work smoothly on both CNNs and Transformers.

Table I shows the comparison between SWAP-Score and 10 other zero-shot proxies. The results demonstrate the superior predictive capability of SWAP-Score on Transformer networks, achieving a Spearman's correlation coefficient of 0.711, outperforming all other existing zero-shot metrics, which are either universal metrics or specially designed for Transformers. Notably, the number of model parameters (#Params) and model FLOPs (#FLOPs) outperform most metrics that are more computationally expensive.

Fig. 7 shows the detailed results of SWAP-Score for each task in the GLUE benchmark. SWAP-Score demonstrates strong correlation with the majority of tasks, except for RTE. To analyse this phenomenon, we investigated the performance correlation at the task level, as shown in Fig. 8. The ground-truth performance between most GLUE tasks is strongly correlated, except for RTE. The RTE task exhibits a much weaker correlation with other GLUE tasks. This behaviour is not due to the proposed metric itself, but reflects the underlying characteristics of the task [32]. First, RTE is considerably smaller in scale than most other GLUE datasets, containing only a few thousand labelled examples. This limited size increases variance in model rankings and makes performance less stable across architectures. Second, the linguistic nature of RTE differs from tasks such as SST-2 or MNLI, it involves fine-grained textual entailment over short sentence pairs, which shares less overlap with the features that benefit other tasks. As a result, models that excel on other GLUE tasks do not necessarily achieve consistent gains on RTE [73].

### C. SWAP-Score for Neural Architecture Search

To further validate the effectiveness of SWAP-Score in a practical scenario, we utilised it for NAS by integrating the regularised SWAP-Score with evolutionary search, referred to

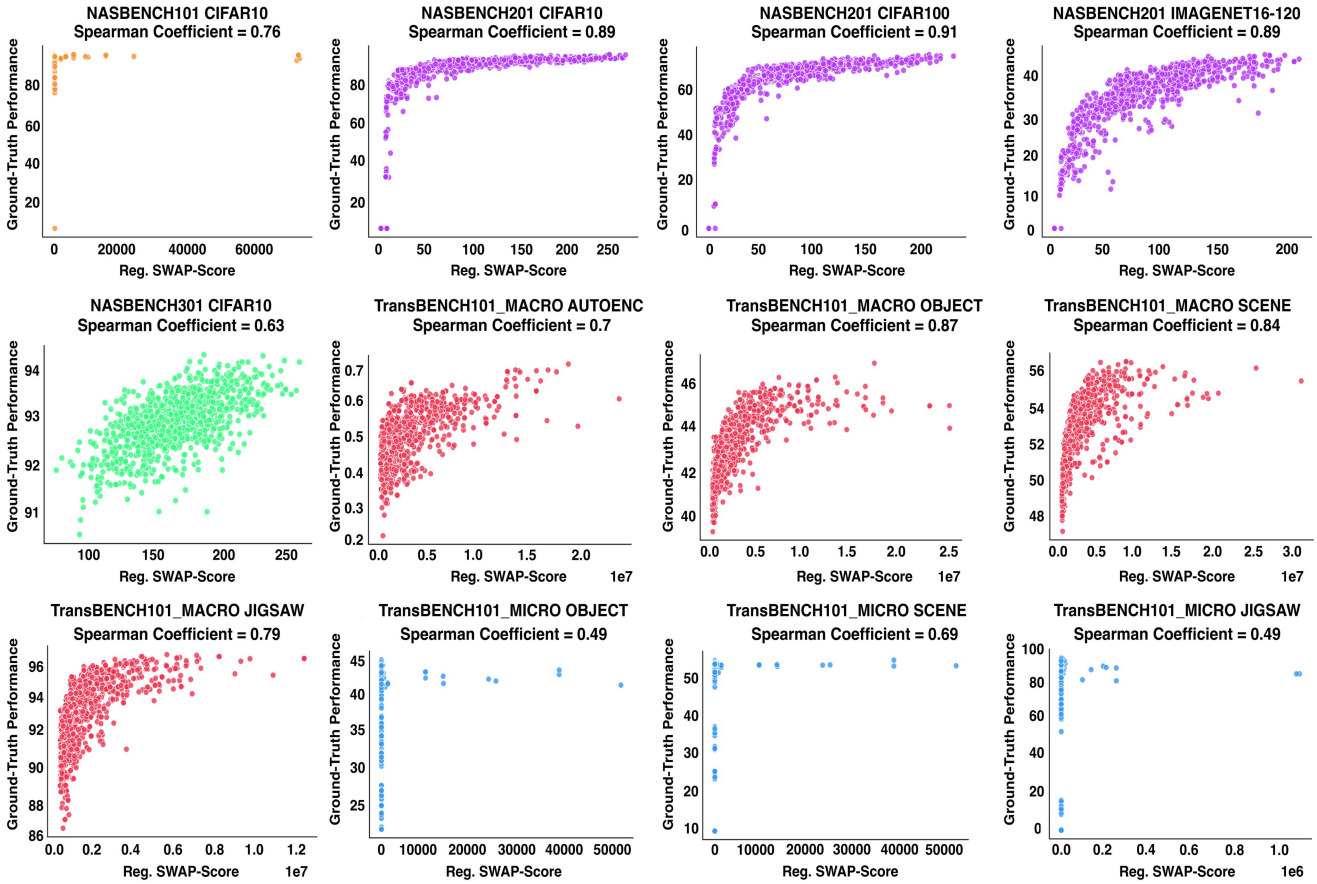


Fig. 6. Scatter plots of regularised SWAP-Score vs. ground-truth performance for architectures from NAS-Bench-101/201/301 and TransNAS-Bench-101. Each subplot corresponds to a vision dataset, and each dot represents a single architecture. Regularised SWAP-Score maintains or improves correlation with true accuracy, showing robustness to variance across tasks.

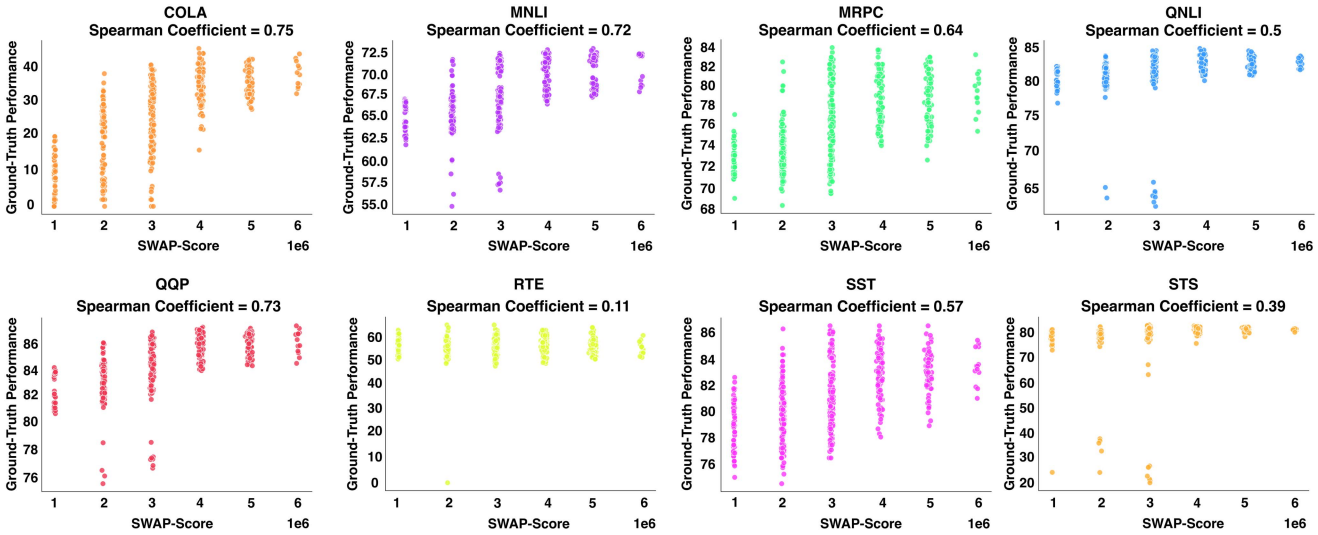


Fig. 7. Scatter plots of SWAP-Score vs. ground-truth performance for Transformer models from FlexiBERT search space on GLUE tasks. Each subplot corresponds to a NLP dataset, and each dot represents a single architecture. Results demonstrate that SWAP-Score generalises beyond CNNs to Transformer architectures, maintaining strong predictive ability on the majority of NLP tasks.

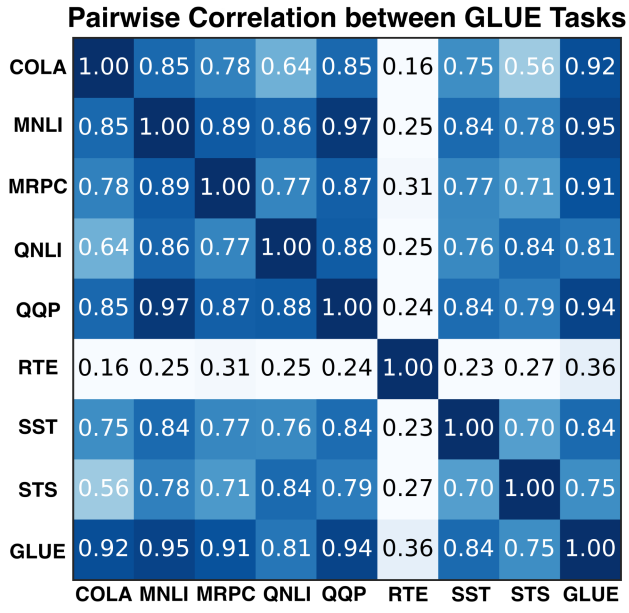


Fig. 8. Pairwise Spearman's correlation among the ground-truth performance of 500 BERT-like Transformers across GLUE tasks. High correlations indicate that models which perform well on one task tend to perform well on others, revealing transferability and consistency of relative model rankings across NLP tasks. While most tasks exhibit strong correlations, the RTE task shows weak alignment with others, indicating limited transferability.

as SWAP-NAS. The DARTS search space was used for the experiments, given its widespread presence in NAS studies, allowing a fair comparison with state-of-the-art methods. For the evolutionary search algorithm, SWAP-NAS is similar to [33], but uses the regularised SWAP-Score as the performance measure. Parent architectures generate possible offspring iteratively in each search cycle, employing both mutation and crossover operations. Unlike many training-based NAS approaches that initiate the search on the CIFAR-10 and later transfer the architecture to the ImageNet, SWAP-NAS conducts direct searches on the ImageNet dataset. This is made feasible due to the high efficiency of the SWAP-Score.

*1) Evolutionary Search Algorithm:* Algorithm 1 details the steps of SWAP-NAS. SWAP-Score replaces the traditional back-propagation training as the performance measurement for the candidate networks (Steps 4 & 12). A tournament strategy is used to sample offspring networks, with half of the networks randomly selected from the population in each search cycle (Step 8). Then, the search algorithm randomly decides whether to perform a crossover operation on the top two candidates of the sampled networks or directly use the best network as the parent (Step 9). The offspring networks are generated by continuously applying mutation to the parent (Step 11). There are two types of mutation: network operation mutation and connectivity mutation, as shown in Fig. 9.

*2) Practical Choice of $\mu$ and $\sigma$:* In practice, their choice can follow two simple regimes. When prior knowledge about the search space is available (e.g., from benchmarks or known ranges of #Params/FLOPs), $\mu$ and $\sigma$ can be set explicitly from that distribution. When prior knowledge is limited or in a search-space agnostic setup, $\mu$ and $\sigma$ can be determined

**Algorithm 1:** SWAP-NAS.

**Require:** Population size $P$, Search cycle $C$, Sample size $S$, $mutation\_times$, Training-free metric *SWAP*

1: population $\leftarrow \emptyset$
2: **while** $population < P$ **do**
3: $model.arch \leftarrow RandomGenerateNetworks()$
4: $model.score \leftarrow SWAP(model.arch)$
5: Add $model$ to the $population$
6: **end while**
7: **for** $c = 1, 2, \ldots, C$ **do**
8: $candidates \leftarrow S$ random samples of $population$
9: $parent \leftarrow$ best in $candidates$ **OR** best from crossover between the best and the second best in $candidates$
10: **while** $mutation\_times$ not reached **do**
11: $child \leftarrow Mutate(parent)$
12: $child.score \leftarrow SWAP(child.arch)$
13: **end while**
14: Add the best $child$ to the $population$
15: Remove the worst from the $population$
16: **end for**

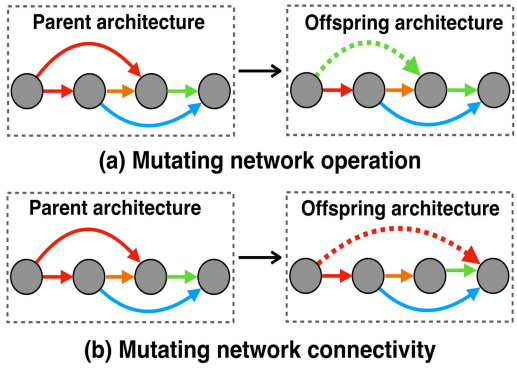


Fig. 9. Illustration of the mutation operators in SWAP-NAS. (a) Operation mutation alters the computational block within the parent network. (b) Connectivity mutation modifies inter-layer connections. Both strategies diversify offspring architectures and improve search coverage.

adaptively from the architectures evaluated during the search process itself. In this case, their values can be updated using the mean and standard deviation of model sizes observed in the search history, at the end of each search epoch. This online adjustment provides a low-cost and flexible heuristic, ensuring that the regularisation remains aligned with the actual search dynamics.

*3) Results on the CIFAR-10:* Table II shows the results of architectures found by SWAP-NAS from the DARTS search space for CIFAR-10. The networks' training strategy and hyper-parameters setting exactly follow the setup in DARTS [18]. Three variations of SWAP-NAS are presented, with different regularisation parameters $\mu$ and $\sigma$. Regardless of these parameters, SWAP-NAS only requires 0.004 GPU days, or 6 minutes. That is 6.5 times faster than the state-of-the-art (TE-NAS). Meanwhile, the architectures found by SWAP-NAS also outperform most previous work. In addition, the capability of model size control is demonstrated. SWAP-NAS-A, with small

TABLE II
PERFORMANCE COMPARISON BETWEEN THE NETWORKS FOUND BY SWAP-NAS AND OTHER METHODS ON CIFAR-10. A LOWER TEST ERROR RATE IS BETTER. † MEANS THE METHOD ADOPTED DARTS SPACE. ⋆ INDICATES THE ORIGINAL PAPER ONLY REPORTED THEIR BEST RESULT.

| | Test Error (%) | Params (M) | GPU Days | Search Method | Evaluation |
|---|---|---|---|---|---|
| PNAS [38] | 3.34±0.09 | 3.2 | 225 | SMBO | Predictor |
| EcoNAS [74]† | 2.62±0.02 | 2.9 | 8 | Evolution | Conventional |
| DARTS [18]† | 3.00±0.14 | 3.3 | 4 | Gradient | One-shot |
| EvNAS [75]† | 2.47±0.06 | 3.6 | 3.83 | Evolution | One-shot |
| RandomNAS [76]† | 2.85±0.08 | 4.3 | 2.7 | Random | One-shot |
| EENA [77] | 2.56⋆ | 8.47 | 0.65 | Evolution | Weights Inherit |
| PRE-NAS [15]† | 2.49±0.09 | 4.5 | 0.6 | Evolution | Predictor |
| ENAS [17] | 2.89⋆ | 4.6 | 0.45 | Reinforce | One-shot |
| FairDARTS [42]† | 2.54⋆ | 2.8 | 0.42 | Gradient | One-shot |
| CARS [78]† | 2.62⋆ | 3.6 | 0.4 | Evolution | One-shot |
| P-DARTS [79]† | 2.50⋆ | 3.4 | 0.3 | Gradient | One-shot |
| TNASP [80]† | 2.57±0.04 | 3.6 | 0.3 | Evolution | Predictor |
| PINAT [81]† | 2.54±0.08 | 3.6 | 0.3 | Evolution | Predictor |
| GDAS [82] | 2.82⋆ | 2.5 | 0.17 | Gradient | One-shot |
| CTNAS [16] | 2.59±0.04 | 3.6 | 0.1+0.3 | Reinforce | Predictor |
| MeCo [46]† | 2.69±0.05 | 4.2 | 0.08 | Pruning | Training-free |
| TE-NAS [19]† | 2.63±0.064 | 3.8 | 0.03 | Pruning | Training-free |
| **SWAP-NAS-A ($\mu$=0.9, $\sigma$=0.9)** † | 2.65±0.04 | 3.06 | 0.004 | Evolution | Training-free |
| **SWAP-NAS-B ($\mu$=1.2, $\sigma$=1.2)** † | 2.54±0.07 | 3.48 | 0.004 | Evolution | Training-free |
| **SWAP-NAS-C ($\mu$=1.5, $\sigma$=1.5)** † | 2.48±0.09 | 4.3 | 0.004 | Evolution | Training-free |
| **SWAP-NAS (Adaptive)** † | 2.53±0.07 | 3.51 | 0.004 | Evolution | Training-free |

TABLE III
PERFORMANCE COMPARISON BETWEEN THE NETWORKS FOUND BY SWAP-NAS AND OTHER METHODS ON IMAGENET. A LOWER TEST ERROR RATE IS BETTER. † MEANS THE METHOD ADOPTED DARTS SPACE.

| | Test Error Top1/Top5 | Params (M) | GPU Days | Search Method | Evaluation | Searched Dataset |
|---|---|---|---|---|---|---|
| ProxylessNAS [41] | 24.9 / 7.5 | 7.1 | 8.3 | Gradient | One-shot | ImageNet |
| DARTS [18]† | 26.7 / 8.7 | 4.7 | 4 | Gradient | One-shot | CIFAR-10 |
| EvNAS [75]† | 24.4 / 7.4 | 5.1 | 3.83 | Evolution | One-shot | CIFAR-10 |
| PRE-NAS [15]† | 24.0 / 7.8 | 6.2 | 0.6 | Evolution | Predictor | CIFAR-10 |
| FairDARTS [42]† | 26.3 / 8.3 | 2.8 | 0.42 | Gradient | One-shot | CIFAR-10 |
| CARS [78]† | 24.8 / 7.5 | 5.1 | 0.4 | Evolution | One-shot | CIFAR-10 |
| P-DARTS [79]† | 24.4 / 7.4 | 4.9 | 0.3 | Gradient | One-shot | CIFAR-10 |
| CTNAS [16] | 22.7 / 7.5 | - | 0.1+50 | Reinforce | Predictor | ImageNet |
| ZiCo [24] | 21.9 / - | - | 0.4 | Evolution | Training-free | ImageNet |
| PINAT-T [81]† | 24.9 / 7.5 | 5.2 | 0.3 | Evolution | Predictor | CIFAR-10 |
| GDAS [82] | 27.5 / 9.1 | 4.4 | 0.17 | Gradient | One-shot | CIFAR-10 |
| TE-NAS [19]† | 24.5 / 7.5 | 5.4 | 0.17 | Pruning | Training-free | ImageNet |
| MeCo [46]† | 22.2 / - | 7.9 | 0.08 | Pruning | Training-free | CIFAR-10 |
| QE-NAS [83]† | 25.5 / - | 3.2 | 0.02 | Evolution | Training-free | ImageNet |
| **SWAP-NAS ($\mu$=25, $\sigma$=25)** † | 24.0 / 7.6 | 5.8 | 0.006 | Evolution | Training-free | ImageNet |
| **SWAP-NAS (Adaptive)** † | 23.3 / 6.8 | 7.3 | 0.006 | Evolution | Training-free | ImageNet |

$\mu$ and $\sigma$ values, generates smaller networks but also suffers a tiny performance deterioration. Conversely, SWAP-NAS-C, with large $\mu$ and $\sigma$, achieves the best error rate but at the cost of a slightly larger network. This capability allows practitioners to find a balance between performance and model size according to the needs of the task. Where SWAP-NAS (Adaptive) reports the results obtained when prior knowledge of the search space is limited. In this setting, $\mu$ and $\sigma$ are adaptively assigned based on the statistical information of previously evaluated architectures, with updates applied during the search process. This heuristic strategy enables end-to-end architecture search in a search-space-agnostic scenario.

*4) Results on the ImageNet:* Table III shows the results on the ImageNet dataset, where the training strategy and hyper-parameters setting are also the same as in DARTS [18]. The search cost of SWAP-NAS here slightly increased to 0.006 GPU days, or 9 minutes. This is still 2.3 times faster than the state-of-the-art (QE-NAS) yet with better performance.

### D. Guidelines for Integrating SWAP With Other Paradigms

While our experiments demonstrate SWAP-Score within an evolutionary NAS framework, the metric can also complement other paradigms. In gradient-based NAS (e.g., DARTS [18], GDAS [82]), SWAP-Score cannot directly substitute for the gradient signal, but it may serve as a pre-screening mechanism to reduce the candidate set before optimisation. In reinforcement learning NAS (e.g., ENAS [17]), SWAP-Score can be employed as a reward signal to quickly evaluate sampled policies before committing to training. For predictor-based NAS, SWAP-Score provides a low-cost pseudo-label that can supplement limited ground-truth architecture–accuracy pairs, thereby improving predictor training.

## V. ABLATION STUDIES

A range of ablation studies are conducted to investigate the impacts of batch size, input dimension and input type on

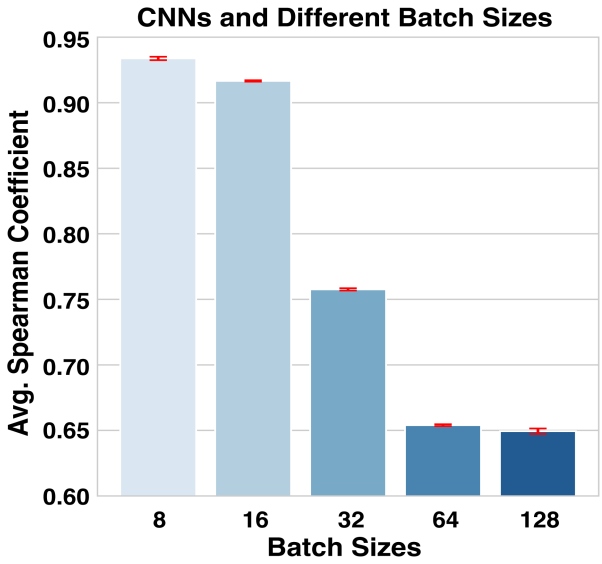


Fig. 10. Spearman's correlation coefficient between SWAP-Score and ground-truth performance of 1000 DARTS CNNs with inputs of different batch sizes. Each setup is repeated 5 times with different random seeds. Each bar shows the average value of 5 runs along with the standard error. The results show a clear tendency for the correlation to decrease as the batch size increase.

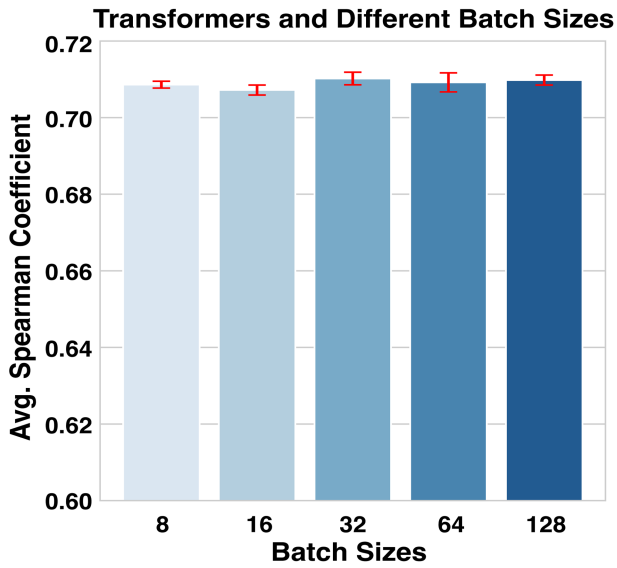


Fig. 11. Spearman's correlation coefficient between SWAP-Score and ground-truth performance of 500 BERT-like Transformers with inputs of different batch sizes. Each setup is repeated 5 times with different random seeds. Each bar shows the average value of 5 runs along with the standard error. The results remain consistent across varying batch sizes.

the SWAP-Score. Similar to Section IV, both ReLU-based and GELU-based networks are involved in these studies. For ReLU-based networks, 1000 cell networks are sampled from the DARTS architecture space and trained for 200 epochs on the CIFAR-10 dataset. For GELU-based networks, this study reuses the 500 BERT-like Transformers as introduced in Section IV-B.

### A. *Impacts of Batch Sizes*

Fig. 10 illustrates the average Spearman's correlation coefficient between the SWAP-Score and the ground-truth performance of 1,000 DARTS CNNs on the CIFAR-10 dataset, over five runs with different random seeds. The results demonstrate a clear tendency for the correlation to decrease as the batch size increases, with the Spearman's correlation coefficient initially reaching 0.933 at a batch size of 8 and dropping to 0.649 at a batch size of 128. Notably, the standard error between each run is very small.

Similarly, Fig. 11 shows the average Spearman's correlation coefficient between the SWAP-Score and the ground-truth GLUE score of 500 BERT-like Transformers. In contrast to the CNN experiment, the correlation coefficients on the Transformers are not significantly impacted by batch size and remain at approximately 0.71.

To further investigate this phenomenon, we examined the number of activation values of the 1000 DARTS CNNs and the 500 BERT-like Transformers. As shown in Fig. 12, the number of activation values for the CNNs are distributed in the range of $10^4$, while for the Transformers, they are distributed in the range of $10^6$. According to (4), the Transformers have much larger $V$ values than the CNNs, making their sample-wise activation patterns more tolerant for larger batch sizes or more input samples, $S$. These results suggest that when calculating

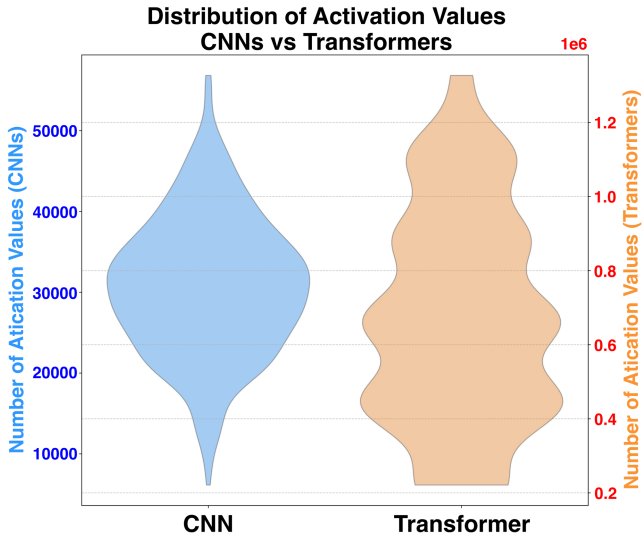


Fig. 12. Distribution of the number of activation values of 1000 DARTS CNNs and 500 BERT-like Transformers. The comparison illustrates that the BERT-like Transformers contain much more activation values than the DARTS CNNs.

SWAP-Score for small neural networks like DARTS CNNs, a smaller batch size yields better performance.

### B. *Impacts of Input Dimensions and Types*

Aside from the batch size of input samples, the impacts of input dimension and input type are also investigated. We set the batch size to 16 and utilised the 1,000 DARTS CNNs with the CIFAR-10 dataset. For input dimensions, the samples are from the CIFAR-10 dataset, with a size starting from $3 \times 3 \times 3$ and gradually increasing to $3 \times 32 \times 32$, the original dimension

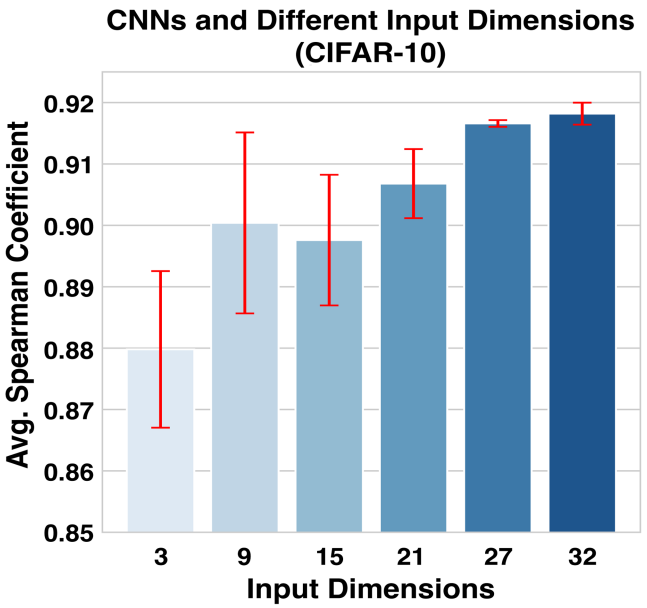


Fig. 13. Spearman's correlation coefficient between SWAP-Score and ground-truth performance of 1000 DARTS CNNs with CIFAR-10 images of different dimensions. Each setup is repeated 5 times with different random seeds. Each bar shows the average value of 5 runs along with the standard error. The results illustrate the sensitivity of SWAP-Score to input image resolution, a larger input dimension yields better correlation.

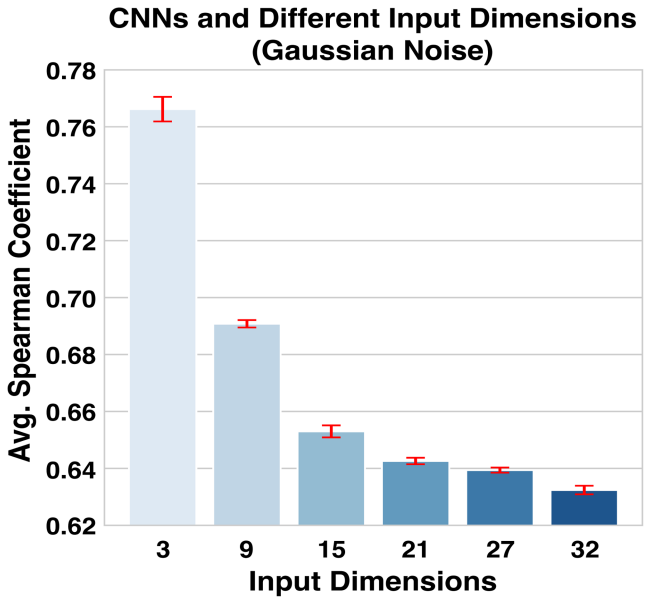


Fig. 14. Spearman's correlation coefficient between SWAP-Score and ground-truth performance of 1000 DARTS CNNs with Gaussian noises of different dimensions. Each setup is repeated 5 times with different random seeds. Each bar shows the average value of 5 runs along with the standard error. Unlike the results in Fig. 13, the Spearman's coefficient decreases as the input dimension increases.

of the CIFAR-10 images. Furthermore, some of the CIFAR-10 images are replaced with Gaussian noise to investigate the impacts of different input types.

As shown in Fig. 13, the Spearman's correlation coefficient gradually increases along with the input dimensions. This further verifies our observation in Section III-B that a larger $V$ offers a higher capacity for distinct patterns, as larger dimensional inputs yield more intermediate values when the architectures of the networks are fixed. In contrast, when the input type is Gaussian noise, the Spearman's correlation coefficient decreases as the input dimension increases, showing in Fig. 14. This indicates that original task data offers more informative activation patterns that align with network performance, whereas random noise lacks the structure necessary to provide meaningful signals.

**Guidelines on Batch Size and Input Dimensions.** Above ablation studies reveal that both batch size and input dimension influence the reliability of SWAP-Score. For CNNs in the DARTS search space, Spearman's correlation decreases as batch size increases (from 0.933 at $S = 8$ to 0.649 at $S = 128$). This is because CNNs have a relatively small number of activation values ($V \approx 10^4$), so larger batches quickly saturate the diversity of activation patterns. In contrast, BERT-like Transformers ($V \approx 10^6$) tolerate larger batch sizes, maintaining stable correlations around 0.71. For small CNNs, use smaller batches (8–16) to preserve sensitivity, while for large-scale models such as Transformers, batch size has little effect and can be set more flexibly.

For input dimensions, higher-resolution task data (e.g., CIFAR-10 at $3 \times 32 \times 32$) improves correlation because larger inputs produce more informative intermediate activations. However, when inputs are replaced with Gaussian noise, increasing dimension reduces correlation, as the activations do not reflect meaningful task structure.

TABLE IV
SPEARMAN CORRELATION COEFFICIENTS OF REGULARISED AND UN-REGULARISED BASELINES. WE APPLY THE REGULARISATION FUNCTION $f(\Theta)$ TO #FLOPS AND #PARAMS TO TEST WHETHER THE GAINS OF REG. SWAP ARISE SOLELY FROM REGULARISATION.

| | NB101 CF10 | NB201 CF10 | NB201 CF100 | NB201 IMGNT | NB301 CF10 |
|---|---|---|---|---|---|
| #FLOPs | 0.37 | 0.69 | 0.71 | 0.67 | 0.43 |
| Reg. #FLOPs | 0.76 | 0.69 | 0.71 | 0.67 | 0.53 |
| #Params | 0.38 | 0.72 | 0.73 | 0.69 | 0.46 |
| Reg. #Params | 0.76 | 0.72 | 0.73 | 0.69 | 0.54 |
| SWAP | 0.47 | 0.79 | 0.81 | 0.76 | 0.58 |
| Reg. SWAP | 0.75 | 0.88 | 0.90 | 0.87 | 0.63 |

### C. Effect of Regularisation on Baselines

To further examine whether the gains of Reg. SWAP arise solely from the Gaussian weighting function $f(\Theta)$, we applied the same regularisation to two simple baselines: #FLOPs and #Params. Table IV reports the results across NAS-Bench-101/201/301 architecture spaces.

We observe that regularisation indeed improves both baselines on certain tasks. For example, on NAS-Bench-101 (CIFAR-10), the Spearman correlation of #Params increases from 0.38 to 0.76 after regularisation, and #FLOPs rises from 0.37 to 0.76. However, on NAS-Bench-201, neither baseline benefits from the regularisation. In contrast, Reg. SWAP remains consistently superior, with correlations of 0.75–0.90 across all benchmarks, outperforming both Reg. #Params and Reg. #FLOPs by a clear margin.

These findings demonstrate that while the regularisation function reduces size bias, the activation pattern expressivity captured by SWAP provides additional predictive strength. In other words, Reg. SWAP benefits from both scale normalisation and structural information, explaining its superior performance over purely regularised size-based baselines.

## VI. Conclusion and Future Work

In this study, we introduced SWAP, Sample-Wise Activation Patterns, and subsequently SWAP-Score, a novel, broadly applicable, and highly effective zero-shot metric based on SWAP. SWAP brings a new perspective for examining the characteristics of networks with different topologies. It can overcome the limitations of standard activation patterns, offering much better generalisation over various domains and tasks, for both CNN and Transformer architectures. More importantly, SWAP provides much stronger correlation with the ground-truth performance than existing SOTA zero-shot or training-free metrics across various computer vision and natural language processing tasks. Additionally, the effectiveness of SWAP-Score can be demonstrated through in an important scenario, NAS. When integrated with an evolutionary search algorithm as a new NAS method, SWAP-NAS, a combination of fast low-cost search and highly competitive performance can be achieved on both CIFAR-10 and ImageNet datasets, outperforming state-of-the-art NAS methods.

In this work, the $Signum$ function is adopted as the indicator function. In the future work we will see whether it can be replaced by a more fine-grained function, which is capable of capturing nuanced information within the activation values. Moreover, as pre-training has become a standard procedure in modern Large Language Models and Large Vision Models, exploring the impacts of pre-train datasets and hyper-parameters in a zero-shot fashion is also a worthy direction.

## References


[1] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *Proc. 4th Int. Conf. Learn. Representations*, 2016.

[2] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1135–1143.

[3] J. Mellor, J. Turner, A. J. Storkey, and E. J. Crowley, "Neural architecture search without training," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 7588–7598.

[4] M. S. Abdelfattah, A. Mehrotra, L. Dudziak, and N. D. Lane, "Zero-cost proxies for lightweight NAS," in *Proc. 9th Int. Conf. Learn. Representations*, 2021.

[5] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio, "On the number of linear regions of deep neural networks," in *Proc. 27th Annu. Conf. Neural Inf. Process. Syst.*, 2014, pp. 2924–2932.

[6] S. Lin et al., "Knowledge distillation via the target-aware transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10915–10924.

[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[8] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015.

[9] D. Silver et al., "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[10] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. 5th Int. Conf. Learn. Representations*, 2017.

[11] P. Ren et al., "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 76:1–76:34, 2022.

[12] C. White et al., "Neural architecture search: Insights from 1000 papers," 2023, *arXiv:2301.08727*.

[13] Y. Sun, H. Wang, B. Xue, Y. Jin, G. G. Yen, and M. Zhang, "Surrogate-assisted evolutionary deep learning using an end-to-end random forest-based performance predictor," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 350–364, Apr. 2020.

[14] W. Wen, H. Liu, Y. Chen, H. H. Li, G. Bender, and P. Kindermans, "Neural predictor for neural architecture search," in *Proc. 16th Eur. Conf. Comput. Vis.*, Glasgow, U.K., 2020, pp. 660–676.

[15] Y. Peng, A. Song, V. Ciesielski, H. M. Fayek, and X. Chang, "PRE-NAS: Evolutionary neural architecture search with predictor," *IEEE Trans. Evol. Comput.*, vol. 27, no. 1, pp. 26–36, Feb. 2023.

[16] Y. Chen et al., "Contrastive neural architecture search with neural architecture comparators," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 9502–9511.

[17] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.

[18] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. 7th Int. Conf. Learn. Representations*, 2019.

[19] W. Chen, X. Gong, and Z. Wang, "Neural architecture search on imagenet in four GPU hours: A theoretically inspired perspective," in *Proc. 9th Int. Conf. Learn. Representations*, 2021.

[20] M. Lin et al., "Zen-NAS: A zero-shot NAS for high-performance image recognition," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Montreal, QC, Canada, 2021, pp. 337–346.

[21] V. Lopes, S. Alirezazadeh, and L. A. Alexandre, "EPE-NAS: Efficient performance estimation without training for neural architecture search," in *Proc. 30th Int. Conf. Artif. Neural Netw. Mach. Learn.*, Bratislava, Slovakia, 2021, pp. 552–563.

[22] J. Mok, B. Na, J. Kim, D. Han, and S. Yoon, "Demystifying the neural tangent kernel from a practical perspective: Can it be trusted for neural architecture search without training?," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, New Orleans, LA, USA, 2022, pp. 11851–11860.

[23] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli, "Pruning neural networks without any data by iteratively conserving synaptic flow," in *Proc. 33rd Annu. Conf. Neural Inf. Process. Syst.*, 2020, pp. 6377–6389.

[24] G. Li, Y. Yang, K. Bhardwaj, and R. Marculescu, "ZiCo: Zero-shot NAS via inverse coefficient of variation on gradients," in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023.

[25] A. Krishnakumar, C. White, A. Zela, R. Tu, M. Safari, and F. Hutter, "NAS-bench-suite-zero: Accelerating research on zero cost proxies," in *Proc. 36th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2022, pp. 28037–28051.

[26] H. Xiong, L. Huang, M. Yu, L. Liu, F. Zhu, and L. Shao, "On the number of linear regions of convolutional neural networks," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 10514–10523.

[27] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter, "NAS-bench-101: Towards reproducible neural architecture search," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 7105–7114.

[28] X. Dong and Y. Yang, "NAS-bench-201: Extending the scope of reproducible neural architecture search," in *Proc. 8th Int. Conf. Learn. Representations*, Addis Ababa, Ethiopia, 2020.

[29] J. Siems, L. Zimmer, A. Zela, J. Lukasik, M. Keuper, and F. Hutter, "Surrogate NAS benchmarks: Going beyond the limited search spaces of tabular NAS benchmarks," in *Proc. 10th Int. Conf. Learn. Representations*, 2022.

[30] Y. Duan et al., "TransNAS-bench-101: Improving transferability and generalizability of cross-task neural architecture search," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5251–5260.

[31] S. Tuli, B. Dedhia, S. Tuli, and N. K. Jha, "FlexiBERT: Are current transformer architectures too homogeneous and rigid?," *J. Artif. Intell. Res.*, vol. 77, pp. 39–70, 2023.

[32] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. 7th Int. Conf. Learn. Representations*, 2019.

[33] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4780–4789.

[34] B. Baker, O. Gupta, R. Raskar, and N. Naik, "Practical neural network performance prediction for early stopping," 2017, *arXiv:1705.10823*.
[35] H. Liang et al., "DARTS+: Improved differentiable architecture search with early stopping," 2019, *arXiv:1909.06035*.
[36] A. Klein, S. Falkner, J. T. Springenberg, and F. Hutter, "Learning curve prediction with Bayesian neural networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017.
[37] M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2820–2828.
[38] C. Liu et al., "Progressive neural architecture search," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 19–34.
[39] R. Luo, X. Tan, R. Wang, T. QinE. Chen, and T. Liu, "Semi-supervised neural architecture search," in *Proc. 33rd Annu. Conf. Neural Inf. Process. Syst.*, 2020, pp. 10547–10557.
[40] H. Shi, R. Pi, H. Xu, Z. Li, J. T. Kwok, and T. Zhang, "Bridging the gap between sample-based and one-shot neural architecture search with BONAS," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1808–1819.
[41] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *Proc. 7th Int. Conf. Learn. Representations*, 2019.
[42] X. Chu, T. Zhou, B. Zhang, and J. Li, "Fair DARTS: Eliminating unfair advantages in differentiable architecture search," in *Proc. 16th Eur. Conf. Comput. Vis.*, Glasgow, U.K., 2020, pp. 465–480.
[43] X. Dong and Y. Yang, "One-shot neural architecture search via self-evaluated template network," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Seoul, South Korea, 2019, pp. 3680–3689.
[44] Y. Xu et al., "PC-DARTS: Partial channel connections for memory-efficient differentiable architecture search," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.
[45] L. Xie et al., "Weight-sharing neural architecture search: A battle to shrink the optimization gap," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–37, 2021.
[46] T. Jiang, H. Wang, and R. Bie, "MeCo: Zero-shot NAS with one data and single forward pass via minimum eigenvalue of correlation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 61020–61047.
[47] A. Jacot, C. Hongler, and F. Gabriel, "Neural tangent kernel: Convergence and generalization in neural networks," in *Proc. 33rd Annu. Conf. Neural Inf. Process. Syst.*, 2018, pp. 8580–8589.
[48] Q. Zhou et al., "Training-free transformer architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10894–10903.
[49] A. Serianni and J. Kalita, "Training-free neural architecture search for RNNs and transformers," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguist.*, 2023, pp. 2522–2540.
[50] R. Pascanu, G. Montufar, and Y. Bengio, "On the number of response regions of deep feed forward networks with piece-wise linear activations," in *Proc. 2nd Int. Conf. Learn. Representations*, 2014.
[51] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 807–814.
[52] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2016, *arXiv:1606.08415*.
[53] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[54] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf
[55] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, Miami, FL, USA, 2009, pp. 248–255.
[56] P. Chrabaszcz, I. Loshchilov, and F. Hutter, "A downsampled variant of imagenet as an alternative to the CIFAR datasets," 2017, *arXiv:1707.08819*.
[57] A. R. Zamir et al., "Taskonomy: Disentangling task transfer learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3712–3722.
[58] B. Zhou, À. Lapedriza, A. Khosla, A. Oliva, and A. Torralba, "Places: A 10 million image database for scene recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 6, pp. 1452–1464, Jun. 2018.
[59] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
[60] J. Turner, E. J. Crowley, M. O'Boyle, A. Storkey, and G. Gray, "BlockSwap: Fisher-guided block substitution for network compression on a budget," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.
[61] X. Ning et al., "Evaluating efficient performance estimators of neural architectures," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 12265–12277.
[62] C. Wang, G. Zhang, and R. Grosse, "Picking winning tickets before training by preserving gradient flow," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.
[63] N. Lee, T. Ajanthan, and P. H. S. Torr, "SNIP: Single-shot network pruning based on connection sensitivity," in *Proc. 7th Int. Conf. Learn. Representations*, 2019.
[64] H. Tanaka, D. Kunin, D. L. Yamins, and S. Ganguli, "Pruning neural networks without any data by iteratively conserving synaptic flow," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 6377–6389.
[65] A. Gokaslan and V. Cohen, "Openwebtext corpus," 2019. [Online]. Available: http://Skylion007.github.io/OpenWebTextCorpus
[66] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training text encoders as discriminators rather than generators," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.
[67] A. Warstadt, A. Singh, and S. R. Bowman, "Neural network acceptability judgments," *Trans. Assoc. Comput. Linguist.*, vol. 7, pp. 625–641, 2019.
[68] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist. - Hum. Lang. Technol.*, 2018, pp. 1112–1122.
[69] B. Dolan and C. Brockett, "Automatically constructing a corpus of sentential paraphrases," in *Proc. 3rd Int. Workshop Paraphrasing*, 2005, pp. 9–16.
[70] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000 questions for machine comprehension of text," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2016, pp. 2383–2392.
[71] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2013, pp. 1631–1642.
[72] D. Cer, M. Diab, E. E. Agirre, I. Lopez-Gazpio, and L. Specia, "SemEval-2017 task 1: Semantic textual similarity multilingual and cross-lingual focused evaluation," in *Proc. 11th Int. Workshop Semantic Eval.*, 2017, pp. 1–14.
[73] T. Vu et al., "Exploring and predicting transferability across NLP tasks," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2020, pp. 7882–7926.
[74] D. Zhou et al., "EcoNAS: Finding proxies for economical neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Seattle, WA, USA, 2020, pp. 11393–11401.
[75] N. Sinha and K. Chen, "Evolving neural architecture using one shot model," in *Proc. ACM Genet. Evol. Computation Conf.*, Lille, France, 2021, pp. 910–918.
[76] L. Li and A. Talwalkar, "Random search and reproducibility for neural architecture search," in *Proc. 35th Uncertainty Artif. Intell. Conf.*, 2020, pp. 367–377.
[77] H. Zhu, Z. An, C. Yang, K. Xu, E. Zhao, and Y. Xu, "EENA: Efficient evolution of neural architecture," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops*, Seoul, South Korea, 2019, pp. 1891–1899.
[78] Z. Yang et al., "CARS: Continuous evolution for efficient neural architecture search," in *Proc. 2020 IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Seattle, WA, USA, 2020, pp. 1826–1835.
[79] X. Chen, L. Xie, J. Wu, and Q. Tian, "Progressive differentiable architecture search: Bridging the depth gap between search and evaluation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1294–1303.
[80] S. Lu, J. Li, J. Tan, S. Yang, and J. Liu, "TNASP: A transformer-based NAS predictor with a self-evolution framework," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 15125–15137.
[81] S. Lu et al., "PINAT: A permutation invariance augmented transformer for NAS predictor," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 8957–8965.
[82] X. Dong and Y. Yang, "Searching for a robust neural architecture in four GPU hours," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1761–1770.
[83] Z. Sun et al., "Entropy-driven mixed-precision quantization for deep network design," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 21508–21520.

**Yameng Peng** (Student Member, IEEE) received the Master of Data Science and PhD degrees in artificial intelligence from the Royal Melbourne Institute of Technology (RMIT University), Melbourne, VIC, Australia, in 2019 and 2024, respectively. His research interests include automated deep learning, natural language processing, and computer vision.

**Andy Song** (Member, IEEE) received the PhD degree from RMIT University, Australia. Dr Song is currently an associate professor of artificial intelligence with the School of Computing Technologies, RMIT University, Australia. He is also with the Centre for Industrial AI Research and Innovation, RMIT and works with a wide range of local and international industry partners for real-world AI adoption in industry sectors such as transport, logistics, smart cities, proptech, argitech, foodtech, and waste management. His research interests include advance evolutionary computation methods for complex tasks, in particular computer vision, optimisation, deep learning, and data analytics.

**Haytham M. Fayek** (Senior Member, IEEE) received the BEng (Hons) and MS degrees in 2012 and 2014, respectively, and the PhD degree from the Royal Melbourne Institute of Technology (RMIT University), Melbourne, VIC, Australia, in 2019. He was a postdoctoral research scientist with Meta, Redmond, WA, USA, from 2018 to 2020. He joined the faculty with RMIT University, Melbourne, VIC, Australia, in 2020. Dr Fayek is currently a senior lecturer with the School of Computing Technologies, RMIT. His research interests include machine learning, deep learning, and machine perception.

**Vic Ciesielski** received the BSc and MSc degrees from the University of Melbourne, Australia, and the PhD degree from Rutgers University, USA. He is currently an associate professor of artificial intelligence with the School of Computing Technologies, RMIT University, Melbourne, VIC, Australia. His research interests include genetic programming, data mining, evolutionary computer vision, deep learning, and evolutionary art.

**Xiaojun Chang** (Senior Member, IEEE) is currently a professor with Future Media Computing Lab, School of Information Science and Technology, University of Science and Technology of China. He was a professor with Australian Artificial Intelligence Institute, University of Technology Sydney, lecturer and senior lecturer with the Faculty of Information Technology, Monash University, Australia, and an associate professor with the School of Computing Technologies, RMIT University. His research focuses on develop structured machine learning models for computer vision and multimedia tasks.